

Aufgabe 1: Monitorkonzept: Ampelsteuerung

20 Punkte



Gegeben sei eine Straßenkreuzung mit den beiden Einbahnstraßen Martinstraße und Jahnstraße sowie den beiden Ampeln A1 und A2.

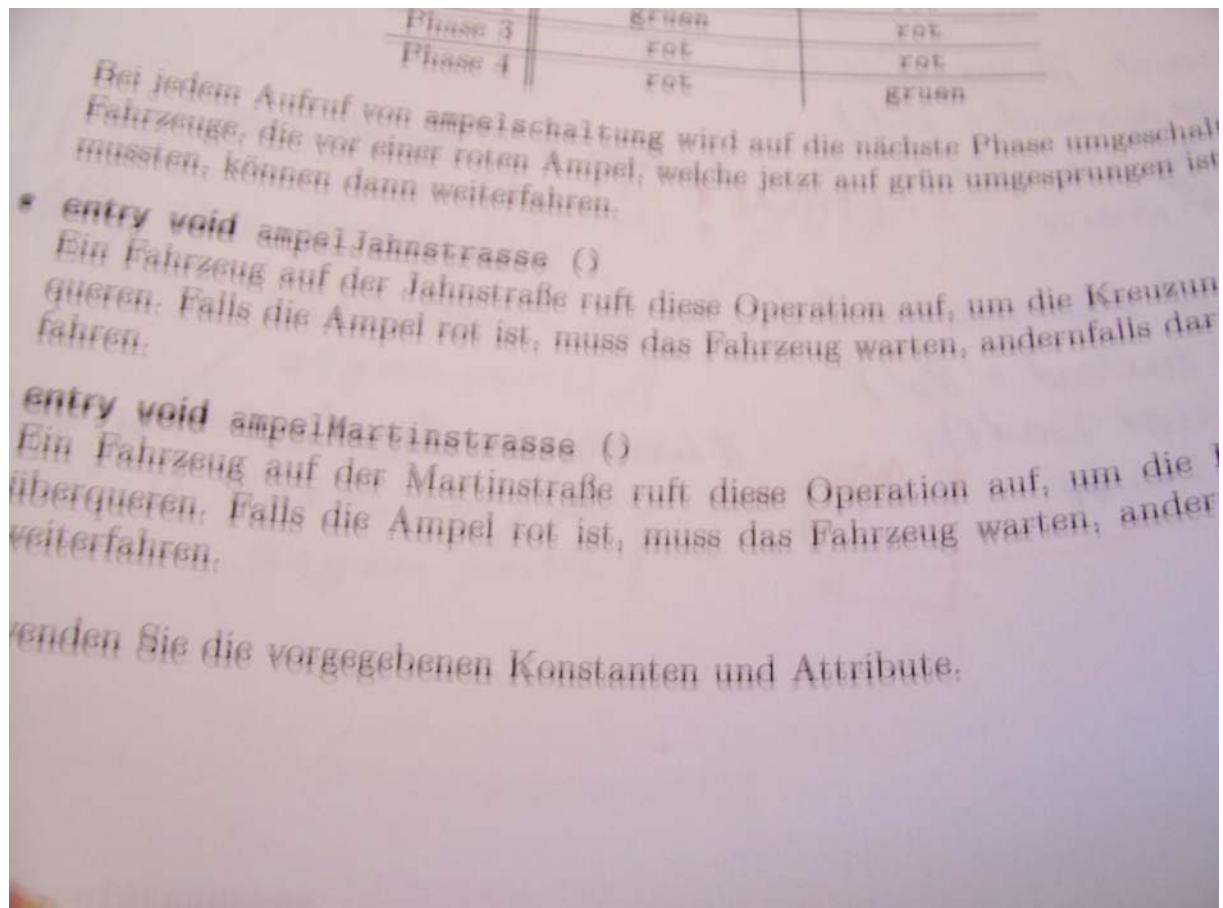
Für die Regelung des Verkehrs auf der Kreuzung soll ein Monitor Ampelsteuerung implementiert werden, der die drei folgenden Monitoroperationen besitzt:

- **entry void ampelschaltung ()**

Diese Operation wird zeitgesteuert von dem Rechner aufgerufen, der die Ampelsteuerung ausführt. Die Ampeln selbst werden in den folgenden vier Phasen geschaltet, die sich dann zyklisch wiederholen (d.h. nach Phase 4 folgt wieder Phase 1):

	A1 (Martinstraße)	A2 (Jahnstraße)
Phase 1	rot	rot
Phase 2	gruen	rot
Phase 3	rot	rot
Phase 4	rot	gruen

Bei jedem Aufruf von ampelschaltung wird auf die nächste Phase umgeschaltet. Alle Fahrzeuge, die vor einer roten Ampel, welche jetzt auf grün umgesprungen ist, warten mussten, können dann weiterfahren.

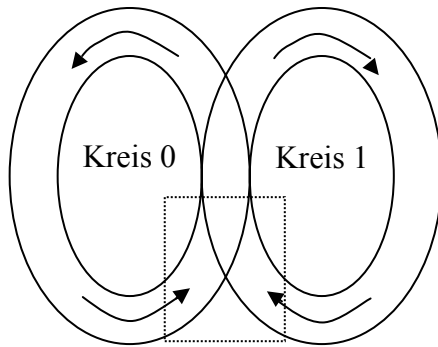


Synchronisationsgraph:



```
monitor Ampelschaltung {
    static final int ROT = 0;
    static final int GRUEN = 1;
    int ampelphase = 1;
    int a1zustand = ROT;
    int a2zustand = ROT;
    //Ihre Objektdeklaration

} //Ampelschaltung
```



Beim Spielen mit ihrer neuen Holzeisenbahn legen Silvio und Gerhard eine Strecke aus zwei Schienenkreisen, die in der Mitte ein gemeinsames Teilstück haben (s. Bild). Damit es keinen Streit gibt, beschließen die beiden, dass ihre Loks immer nur abwechselnd dieses gemeinsame Schienenstück befahren dürfen.

Entwerfen Sie unter Verwendung von Semaphoren eine Steuerung für die beiden Loks. Beachten Sie dabei, dass die Loks unterschiedlich schnell sind. Eine Lok, die die Weiche erreicht, jedoch noch nicht an der Reihe ist, muss warten, bis die andere Lok das Mittelstück verlässt. Andererseits soll die Lok ohne zu warten weiterfahren, wenn sie die Weiche erreicht und klar ist, dass das Schienenstück für sie frei ist. Verwenden Sie kein aktives Warten!

Die Loks führen die folgende Endlosschleife aus:

```

While (true) {
    //fahren
    enterLok0 ();
    //kA0: Lok 0 im mittleren Teilstück
    exitLok0();
}
bzw.
While (true) {
    //fahren
    enterLok1 ();
    //kA1: Lok 1 im mittleren Teilstück
    exitLok1();
}
    
```

Geben Sie den Synchronisationsgraphen und den Code für die Synchronisation an unter der Annahme, dass Lok 0 zu Beginn das Mittelstück zuerst befahren darf.

Synchronisationsgraph:

Code:

Aufgabe 3: Prozessverwaltung**20 Punkte**

Gegeben sei ein System mit 6 Prozessen mit den Identifikationen 0 bis 5 und einem Monitor `mon`, in dem eine Condition-Variable `cond` deklariert ist. In der Prozessverwaltung ist das Monitorkonzept (3. signal-Variante) sowie ein Zeitscheibenverfahren mit Round-Robin-Strategie implementiert. Den aktuellen Zustand erkennen Sie in der aktuellen Tabelle, wobei der Inhalt der FIFO-Warteschlangen in Listenschreibweise von links beginnend (d.h. das älteste Element steht links) dargestellt wird. Die in der ersten Spalte der Tabelle stehenden Schnittstellenoperationen der Prozessverwaltung werden nacheinander aufgerufen (vom jeweils laufenden Prozess) und ausgeführt, wodurch der Zustand geändert wird.

Tragen Sie in die Tabelle jeweils den zwischen zwei Aufrufen herrschenden Zustand ein und unterstreichen Sie jeweils den Prozess, der gerade die Monitorkontrolle besitzt.

In der Tabelle steht `r_p` für `running_process`.

Zustand der Prozessverwaltung					
R_p	Ready Queue	Monitorwarteschlangen- und anzeige			
		Entrance	Urgent	Cond	Busy
5	4	0,3	-	2,1	TRUE
Time interrupt handler					
Wait (mon, cond)					
Enter monitor (mon)					
signal (mon, cond)					
Exit_monitor (mon)					
Time interrupt handler					

Aufgabe 5: Speicherverwaltung**20 Punkte**

- a) Die Hauptspeicherverwaltung teilt Speicherbereiche variabler Länge zu und führt über die freien Bereiche in der Freispeicherliste `Buch`. Der Listenkopf `free` benennt die Speicheradresse des Listenanfangs. In jedem freien Block stehen dessen Länge sowie die Adresse des nächsten freien Blocks. Zu Beginn sei in der Freispeicherliste der gesamte Speicher als zusammenhängender Bereich der Länge 100. Freigegebene Speicherbereiche werden ggf. an das Ende der Freispeicherliste angehängt. Reste, die bei einer Anforderung übrig sind, bleiben in der Freispeicherliste an der Stelle des ausgewählten Speicherbereichs.

Falls eine Anforderung erfüllt ist, skizzieren Sie bitte jeweils die Speicherbelegung sowie die Freispeicherliste (free) nach deren Bearbeitung. Anderenfalls geben Sie bitte „nicht erfüllbar“ an.

a) Es wird die First-Fit-Strategie verwendet und die folgenden Speicheranforderungen und –Freigaben treffen ein:

1. A1: anfordern (20)
2. A2: anfordern (20)
3. A3: anfordern (30)
4. freigeben der Anforderung A1
5. A4: anfordern (10)
6. freigeben der Anforderung A3
7. A5: anfordern (50)

Zu Beginn:

0		0	
10	Len = 100	10	
20	Next = -	20	
30		30	
40		40	
50		50	
60		60	
70		70	
80		80	
90		90	
100		100	
	Free = 0		Free = 0

1.

Das ganze dann bis 7. .

b) Es wird die Best-Fit-Strategie verwendet und die folgenden Speicheranforderungen und –Freigaben treffen ein:

1. A1: anfordern (20)
2. A2: anfordern (20)
3. A3: anfordern (30)
4. freigeben der Anforderung A1
5. A4: anfordern (10)
6. freigeben der Anforderung A3
7. A5: anfordern (50)

Zu Beginn:

1.

Das ganze dann bis 7. .

0		0	
10	Len = 100	10	
20	Next = -	20	
30		30	
40		40	
50		50	
60		60	
70		70	
80		80	
90		90	
100		100	
Free = 0		Free = 0	

~~(X)~~ (6 Punkte)

Ein Prozess belegt vier Seiten im virtuellen Adressraum und ihm stehen zw
 cheln (K1, K2) im Hauptspeicher zur Verfügung, die zu Beginn beide frei sin
 werden der Reihe nach die in den Tabellen angegebenen Seitenzugriffe durch
 Führen Sie in den Tabellen darüber Buch, welche Kachel jeweils welche Seite
 und geben Sie jeweils an, wieviele Seitenfehler auftreten, falls die folgende
 gerungsstrategien eingesetzt werden.

2	1	2	+
3	2	2	+
1	3	1	+
2	2	1	+
4	2	1	+
1	2	1	+
2	2	1	+
3	2	1	+
1	1	3	+
2	1	2	+
4	1	2	+

4 Einlagen!

1	1	1	+
2	1	1	+
3	1	1	+
1	1	1	+
2	1	1	+
4	1	1	+
1	1	1	+
2	1	1	+
3	1	1	+
1	1	1	+
2	1	1	+
4	1	1	+

Anz. Seitenfehler = 10

Anz. Seitenfehler = 9

(4 Punkte)

Welche Vorgänge laufen ab, wenn ein Prozess auf eine Variable zugreift, die auf einer Seite liegt, die sich gerade nicht im Hauptspeicher befindet?

Eine der Seiten die sich im Hauptspeicher befindet wird entsprechend der vereinbarten Strategie ausgelagert und die notwendige Seite wird eingelagert; wie?

Wie werden die Vorgänge angesteuert?
Was passiert mit dem Prozess dabei? wie?

Aufgabe 5: Dateisystem – Dateischutz**20 Punkte****(a) (15 Punkte)**

In ferner Zukunft werden Klausuren der BA direkt an Computern geschrieben. Der Professor stellt hierzu eine Datei "klausur.text" mit den Aufgabenstellungen auf den zentralen Server auf dem jeder Student der Veranstaltung (nehmen Sie an, es gibt hier nur drei!) während des Klausurtermins eine persönliche Zugangskennung (stud1, stud2, stud3) besitzt. Student stud1 schreibt seine Lösung in die Datei stud1.text, Student stud2 in stud2.text und Student stud3 in stud3.text. Sie dürfen davon ausgehen, dass auf dem System keine weiteren Benutzerkennungen existieren.

Es sollen die folgenden Regeln gelten:

- (1) Studenten dürfen die Aufgabenstellung lesen, aber nicht verändern oder löschen.
- (2) Der Professor darf die Lösungen der Studenten lesen, aber nicht verändern.
- (3) Kein Student darf die Lösungen eines anderen Studenten lesen, verändern oder löschen.
- (4) Jeder Student darf seine eigene Lösung lesen und verändern.
- (5) Der Professor darf die Aufgabenstellung lesen, verändern und löschen.

(i) (10 Punkte)

Entwerfen Sie den Zugriffsschutz für ein Standard-Unix-Betriebssystem. Nehmen Sie dabei an, dass die Aufgabenstellungen und alle Lösungen in seinem Verzeichnis /home/bsk/ abgespeichert sind. Der Professor hat neben der Aufgabenstellung bereits die drei (noch leeren) Lösungsdateien angelegt; er ist daher ihr Besitzer. Falls keine Lösung möglich sein sollte, begründen Sie wieso nicht.

(a) (3 Punkte)

Ordnen Sie eine Lösung des Problems an, wenn Zugriffsstrukturen (AOL) verwendet werden. Falls keine Lösung möglich ist, begründen Sie warum nicht. Markieren Sie die folgenden Zugriffsstrukturen an: Inset (h), schreiben (k), löschen (l), auslesen (r).

Lesen: Prof. r, Stud. 1. r, Stud. 2. r, Stud. 3. r

Schreiben: Prof. k, Stud. 1. k, Stud. 2. k, Stud. 3. k

Löschen: Prof. l, Stud. 1. l, Stud. 2. l, Stud. 3. l

Gesamt: Prof. r, Stud. 1. r, Stud. 2. r, Stud. 3. r

Gesamt: Prof. k, Stud. 1. k, Stud. 2. k, Stud. 3. k

Gesamt: Prof. l, Stud. 1. l, Stud. 2. l, Stud. 3. l

(b) (3 Punkte)

Welche Vorteile bietet die Verwaltung von sogenannten Runs im NTFS gegenüber den entsprechenden Datenstrukturen in den Inodes im Standard Unix-Filesystem? Erläutern Sie beide Konzepte kurz!

1. Stud. 1. text, Stud. 2. text, Stud. 3. text, Prof. r, d, Stud. 1. r, d, Stud. 2. r, d, Stud. 3. r, d

(b) (5 Punkte)

Welche Vorteile bietet die Verwaltung von sogenannten Runs im NTFS gegenüber den entsprechenden Datenstrukturen in den Inodes im Standard Unix-Filesystem? Erläutern Sie beide Konzepte kurz!

Inodes sind von der Größe her sehr beschränkt; Man kann nur 3 Dateidirektionen eintragen; Runs sind dagegen variabel, da mehrere Runs auf verschieden große Teile zeigen können. das reicht noch nicht!

